

CSE 590 writeup

Jerry Zhu

April 2026

1 Narrative

The field of formal verification of programs is well studied in the domain of programming languages, with roots dating back to the 1960s. Its central goal is to ensure that a program never reaches a "bad" state (e.g. two nodes hold a lock or two leaders are elected at once).

In the beginning, there was Hoare logic, which describes how code changes the state of a running program. Using Hoare logic, we can show that a property holds initially and is preserved by every possible transition (change in state). Properties that satisfy these two conditions are called inductive invariants. Importantly, the safety property is not itself typically inductive, so to prove it, it's often necessary to find a stronger invariant (i.e. one that implies safety).

Hoare logic is typically used to reason about sequential programs, where there is only one thread of execution (and thus a single sequence of states). However, most programs used in production do things concurrently and fall into the categories of simple parallel programs or parameterized systems. Simple parallel programs have a bounded number of concurrent executing processes with shared state, so we have to consider nondeterministic interleavings. Parameterized systems allow an unbounded number of concurrent processes, so we often have to find invariants that relate multiple processes in a uniform way to show correctness for any system size.

Because of this, discovering invariants is extremely tedious and time consuming. Thus, one major question was whether we could automatically discover invariants instead of writing them by hand. The path that this field has taken to answer this question is examined further below.

In 1976, Owicki and Gries extended earlier ideas from sequential program verification (specifically Hoare logic) to provide a framework to reason about the correctness of parallel programs [4]. The basic idea was to reason about each process sequentially and then show that interactions between processes did not invalidate the correctness of any single process [4].

The next logical step was to move from simply parallel programs to programs that could have any number of participating processes (parameterized systems). In 2001, Pnueli et al. showed that even when the safety property is easy to state, proving it often requires additional "invisible invariants" that are not part of the original problem specification [5]. This was an important finding because it showed that the main difficulty is not in checking a proposed invariant, but rather generating plausible invariants in the first place [5].

From there, the question was how to go from manually discovering these invisible invariants to automatically learning them. In 2014, Garg et al. introduced the ICE (Implication CounterExample) framework to do just this [1]. It uses a learner that proposes candidates to a teacher, who then responds with an accept, counterexample, or an implication (one state

must be included if another is included) [1]. This allowed systematic guessing rather than using human intuition to find invariants.

Two years later, in 2016, Garg et al. built on this to show that decision trees could be used as a practical representation for learned invariants [2]. Their approach took predicates over program states as input features and used labeled examples together with implication counterexamples to learn Boolean combinations of those predicates [2]. This mattered because it turned the general ICE idea into something more implementable and interpretable, as a learned decision tree could be translated back into a logical formula and checked for inductiveness by a verifier [2]. Together, these two papers marked a shift in the field from recognizing invariant discovery as the main obstacle to developing concrete learning-based techniques for overcoming it.

In 2019, Ma et al. extended this work more directly to the distributed protocol context with I4 [3]. While the earlier methods showed how invariants could be synthesized in general, I4 focused on the fact that distributed protocols pose additional challenges, since correctness properties often depend on relationships between many nodes and messages [3]. This was important because it showed that the ideas behind automatic invariant learning were not limited to small or generic verification tasks, but could be adapted to the more structured and difficult domain of distributed protocols [3].

Finally, in 2021, Yao et al. published DistAI, which applied data-driven invariant learning specifically to distributed protocols [6]. DistAI built on the earlier insight that invariant discovery is the main obstacle in verification, but focused even more directly on the features that make distributed systems difficult, such as nondeterminism, symmetry, and the need to reason about relationships across many participating nodes [6]. Rather than relying only on manually designed proof strategies, DistAI used generated data from protocol executions to help infer candidate invariants [6]. This was significant because it showed that learning-based verification could move beyond general frameworks and be tailored to the structure of distributed protocols themselves [6].

Much progress has been made in the last 60 years, but open questions remain. One major challenge is how to learn richer invariants that go beyond simple Boolean combinations of hand-designed predicates, since many distributed protocols require quantified or highly relational properties. A related question is how much these systems can reduce their dependence on human guidance, as many current methods still rely on carefully chosen predicates or protocol-specific structure. Another challenge is scalability: methods that work well on small benchmark protocols may become much less effective on larger and more realistic systems with more complicated message patterns or failure behaviors. Finally, there is also the question of generalization, namely whether invariants learned from small instances of a protocol can reliably extend to systems with many participating nodes. These questions suggest that, while learning-based methods have significantly advanced the field, fully automatic verification of distributed protocols remains an active area of research.

2 Annotated References

- **An Axiomatic Proof Technique for Parallel Programs I** [4]

- Problem Addressed

How to formally prove the correctness of parallel programs. Specifically, testing will miss bugs and Hoare logic is insufficient to handle reasoning about programs with the added complexities of nondeterministic interleavings and interactions between concurrent processes.

- Solution

*Extend Hoare logic by adding **cobegin** ... **coend** for parallel composition and **await B then S** for synchronization. To prove correctness, prove each process as if it were sequential. Then, prove that each process is **interference-free**, i.e., that the other processes do not invalidate assertions used in a process's proof.*

- Authors' Evaluation of Solution

The authors define their new rules formally and provide examples of usage in proofs of standard concurrent examples (e.g. Findpos, producer-consumer algos). They also discuss how to show that a concurrent program does not deadlock and terminates by providing some "sufficient" conditions for these properties.

- Limitations and Future Work

- * *Since this work was published in 1976, there is no mention of performance or automation.*
- * *The authors mention that **await B then S** assumes that we can atomically do 1) the check on S and 2) execute S in its entirety. It is simply meant to be a simple way to represent synchronization mechanisms, not an actual thing that should be implemented in practice.*

- **Automatic deductive verification with invisible invariants** [5]

- Problem Addressed

How to automatically verify invariance properties of a distributed system. Specifically, the authors address bounded-data systems, which can have any number of processes, but each process does not have a state space that grows with the total number of processes.

- Solution

The authors first prove that for bounded-data systems, we can bound the states we need to check to show preservation of an invariant to a small, finite set. They then propose an automatically compute candidate auxiliary invariants by checking this small finite set of states.

- Authors' Evaluation of Solution

They give a theoretical evaluation for the proof mentioned above. They give examples of using their methods to verify protocols (e.g. German's cache coherence protocol) and quantitative speedups from using their method. They also check their method against known bad cache protocols.

- Limitations and Future Work
 - * *Their invariant construction method is only a heuristic, so it may fail to prove a true invariant.*
 - * *The paper only deals with the safety property (no liveness, fairness, etc.).*
- **ICE: a robust framework for learning invariants [1]**
 - Problem Addressed

When learning invariants, inductiveness creates the following issue: when a candidate invariant fails because a state in the invariant transitions to a state outside, should we exclude the first state or include the second? Typical invariant learners only use positive and negative examples to represent this situation, forcing the teacher to commit to one direction even though either might be compatible with a correct invariant.
 - Solution

The authors introduce ICE, which adds in implications in addition to the examples and counterexamples. This allows the teacher to communicate inductiveness failures without having to commit to an arbitrary decision. They then give concrete algorithms to use ICE under two settings.
 - Authors' Evaluation of Solution

The authors formally define ICE and prove convergence results for a few algorithms. They also benchmarked their learner (speed and correctness).
 - Limitations and Future Work
 - * *The numerical learner only searches over template-based formulas, so it might miss invariants*
- **Learning invariants using decision trees and implication counterexamples [2]**
 - Problem Addressed

How to adapt decision tree learning to ICE framework?
 - Solution

The authors extend top-down decision-tree learning so that it can handle positive examples, negative examples, and implication counterexamples simultaneously. Their learner classifies implication endpoints as it builds the tree, propagates those choices along implication chains to stay consistent. They also introduce a modified information gain metric that factors in implications.
 - Authors' Evaluation of Solution

The authors prove that their generic ICE decision tree construction always terminates and produces a tree consistent with the ICE sample. Experimentally, they implemented the learner and benchmarked it against CPAchecker. Their tool was the only one to find adequate invariants for all benchmarks and that their model scales much better.

- Limitations and Future Work
 - * *The convergence guarantee only applies when the target invariant is expressible using the chosen attributes*
 - * *The paper mainly studies invariants over numerical and Boolean attributes for relatively small benchmark programs*
 - * *The authors mention adding pruning to the decision tree as a future direction*
- **Automatic deductive verification with invisible invariants [3]**
 - Problem Addressed

Current methods to find invariants for distributed protocols require lots of human investment. The authors want to find an automatic way to do it.
 - Solution

The authors introduce I4. The key observation is that the basic structure of inductive invariants is independent of system size. They create a small, finite instance of the protocol, generalize it, and then check the final invariant using Ivy. If it fails, try larger instance size or prune clauses.
 - Authors’ Evaluation of Solution

The authors test I4 on 7 distributed protocols. It proves all correct with no manual effort, with runtime between 1 and 10.5 minutes.
 - Limitations and Future Work
 - * *I4 cannot check Paxos because of memory constraints*
 - * *The generalization algorithm only supports universal quantifiers (not existential)*
 - * *Still need human effort to guess the initial finite instance size*
- **DistAI: Data-Driven automated invariant learning for distributed protocols [6]**
 - Problem Addressed

I4 depends on checking a single small instance (may not generalize).
 - Solution

The authors introduce DistAI, which learns inductive invariants in three stages. In stage 1, they run the protocol on various instance sizes and project the samples into subsamples matching the candidate formula size. In stage 2, they search for the strongest candidate invariants that hold across all subsamples. In stage 3, they use Ivy to check the candidates, weakening if something fails.
 - Authors’ Evaluation of Solution

The authors prove three theorems: 1) the enumerated invariants form the strongest possible set 2) monotonic weakening never overshoots the correct invariant 3) DistAI is guaranteed to find any existential-free inductive invariant in finite time. They benchmarked against I4 and Fol-IC3, with speedup up to 100x.
 - Limitations and Future Work

- * *Only supports universal quantifiers (not existential)*
- * *Sampling phase can be slow when action argument spaces are sparse*

References

- [1] Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. Ice: a robust framework for learning invariants. In Armin Biere and Roderick Bloem, editors, *Computer Aided Verification*, pages 69–87, Cham, 2014. Springer International Publishing.
- [2] Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. *SIGPLAN Not.*, 51(1):499–512, January 2016.
- [3] Haojun Ma, Aman Goel, Jean-Baptiste Jeannin, Manos Kapritsos, Baris Kasikci, and Kareem A. Sakallah. I4: incremental inference of inductive invariants for verification of distributed protocols. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP ’19, page 370–384, New York, NY, USA, 2019. Association for Computing Machinery.
- [4] Susan Owicki and David Gries. An axiomatic proof technique for parallel programs i. *Acta Informatica*, 6(4):319–340, Dec 1976.
- [5] Amir Pnueli, Sitvanit Ruah, and Lenore Zuck. Automatic deductive verification with invisible invariants. In Tiziana Margaria and Wang Yi, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 82–97, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [6] Jianan Yao, Runzhou Tao, Ronghui Gu, Jason Nieh, Suman Jana, and Gabriel Ryan. DistAI: Data-Driven automated invariant learning for distributed protocols. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 405–421. USENIX Association, July 2021.

8. Implementation Description (Final)

I will implement in OCaml a small system that uses Z3 to automatically learn inductive invariants for the mutex protocol. I will first model the protocol as a state machine with its transitions and the safety property encoded as logical constraints. Since safety properties are not typically inductive by themselves, the system will have to automatically learn auxiliary invariant(s) that imply safety and are preserved across transitions.

For the system itself, I will have to create a method for the system to generate candidate invariants to test from a fixed set of predicates over the program state. The predicate set will be fairly simple, but it should capture enough relationships between variables to allow the system to find a good auxiliary invariant (since invariant expressiveness is limited by the set of predicates). Then, each candidate will be checked with Z3 for inductiveness (i.e. checking initial validity and preservation across transitions).

To systematize the search, I'll implement an ICE-style learning loop where the learner will propose candidate invariants (as described above), while Z3 will return positive/negative examples and implication counterexamples to capture inductiveness violations. Then, as an optimization, I'll train a decision tree over the predicate space (similar to Garg et al. 2016). The resulting tree will then be translated into a logical formula to be checked for inductiveness itself.

I will check the mutex protocol and use a predicate set consisting of simple Boolean relations over process state (e.g. ownership and mutex conditions). I will evaluate the system on the mutex protocol by comparing 1) time to find an inductive invariant that proves safety and 2) number of explored candidates against a randomized enumeration method.